

Kaggle TensorFlow Speech Recognition Challenge

Paweł Pozorski Paweł Florek

Warsaw University of Technology

May 5, 2026

- 1 Problem and Setup
- 2 Results by Phase
- 3 Takeaways

- Dataset: Kaggle TensorFlow Speech Recognition (Google Speech Commands v1 derivative).
- Fixed 12-class setup: 10 commands + `__unknown__` + `__silence__`.
- Strict 4-phase funnel: feature selection, optimization tuning, architecture comparison, open-set strategy comparison.
- Three fixed seeds for every configuration for stability estimates.

Dataset Splits and Evaluation Protocol

- **Phases 1–3:** command-only small splits (`train_small`, `valid_small`, `test_small`).
- **Phase 4:** extended 12-class splits with explicit unknown/silence handling.
- **Seeds:** 3 fixed seeds (0, 42, 2003) for each configuration.
- **Selection rule:** preserve command quality first, then optimize non-command robustness.

Stage-Wise Architectures and Approaches

- **Stage 1 (Feature selection):** comparison on **ConvNeXt + xLSTM** across mel/mfcc/pcen/specaugment pipelines.
- **Stage 2 (Global tuning):** same proxy families with **scheduler + weight decay** sweep (cosine warmup vs reduce-on-plateau).
- **Stage 3 (Architecture comparison):** full comparison of **ConvNeXt, AST, MLP-Mixer** (SSAMBA/xLSTM kept as planned, not executed).
- **Stage 4 (Open-set handling):** top backbones retrained with **flat multiclass, sampling control, loss reweighting, two-stage**.

Feature strategies (Stage 1)

- **Mel-spectrogram:** log-mel time-frequency baseline representation.
- **High-temporal mel:** shorter window/hop for denser time sampling.
- **MFCC:** DCT-compressed mel representation with lower dimensionality.
- **PCEN:** adaptive per-channel normalization emphasizing transients.
- **Mel + SpecAugment:** mel features with time/frequency masking during training.

Model families

- **ConvNeXt (CNN)**: convolutional hierarchy for local time-frequency pattern extraction, with modern architectural improvements (transformer approach).
- **AST (Transformer)**: self-attention over spectrogram patches for global context modeling.
- **MLP-Mixer**: MLP-only token/channel mixing without convolution or attention blocks.

Planned but unused in executed Stage 3

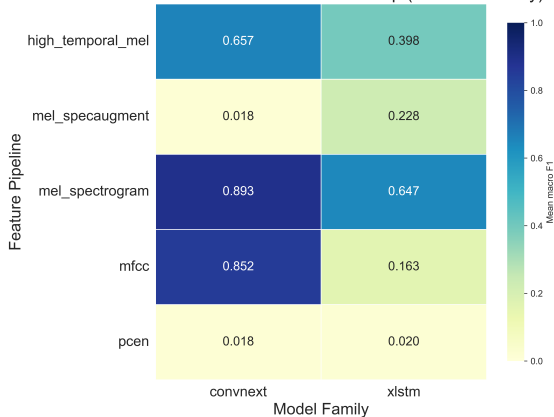
- **xLSTM**: recurrent sequence model, used in early stages and kept in repository scope, dropped due to poor performance.
- **SSAMBA/Mamba**: selective state-space sequence model, implemented but not part of executed Stage 3 runs due to computational constraints.

Open-set methods (Phase 4)

- **Flat multiclass:** one classifier over 12 labels.
- **Sampling control:** weighted batch sampling with predefined class priors.
- **Loss reweighting:** class-weighted cross-entropy to rebalance training signal.
- **Two-stage detector:** binary command/non-command gate plus specialist branches.

Phase 1: Feature Pipeline vs Model Family

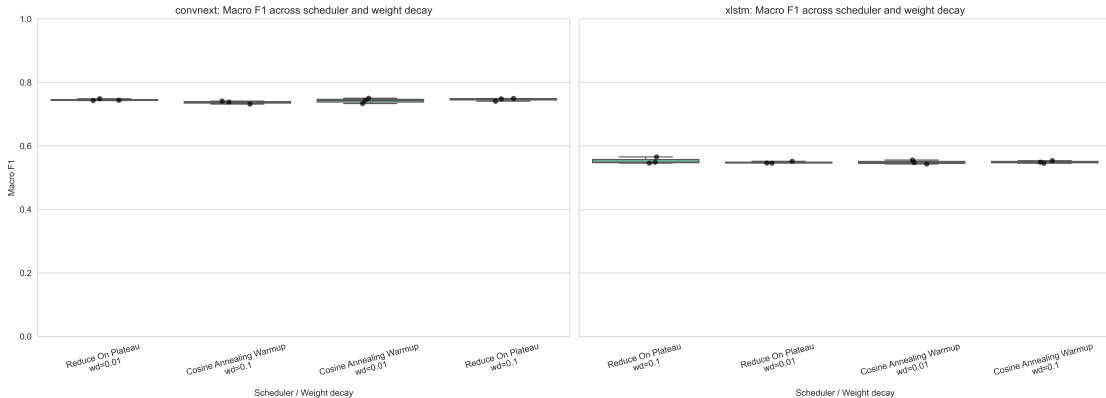
Phase 1: Feature and Model Heatmap (commands only)



Mel-based front ends (including MFCC) are strongest overall; xLSTM underperforms expectations and is dropped later.

Phase 2: Regularization Sensitivity

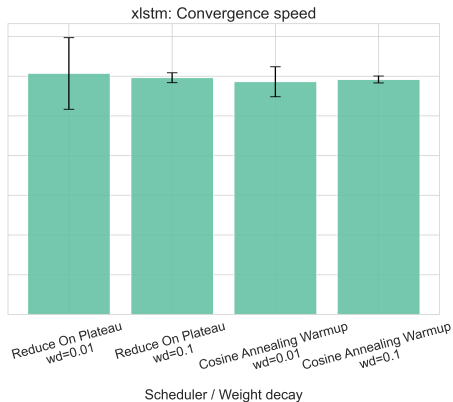
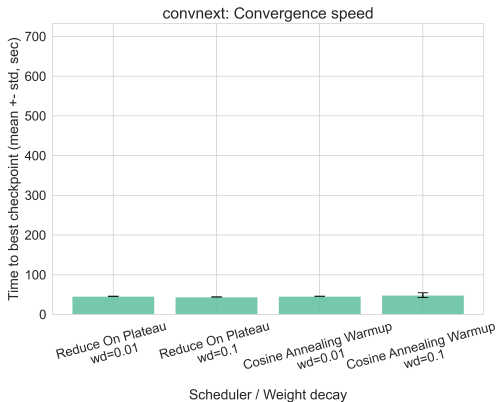
Phase 2: Regularization Sensitivity (distribution across seeds)



Most optimizer/scheduler settings are close in F1.

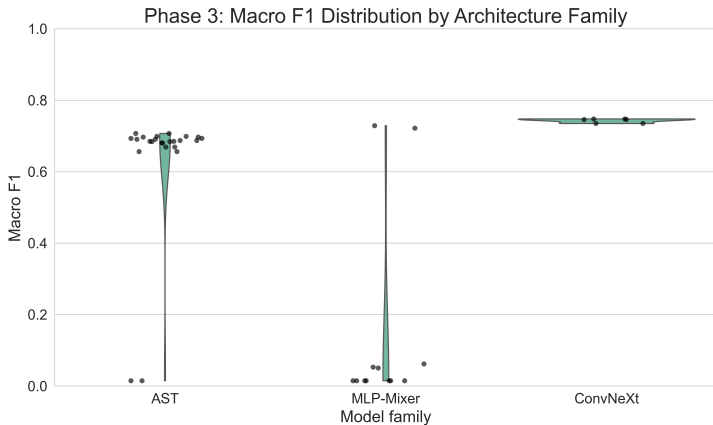
Phase 2: Convergence Speed

Phase 2: Wall time to best validation (sec)



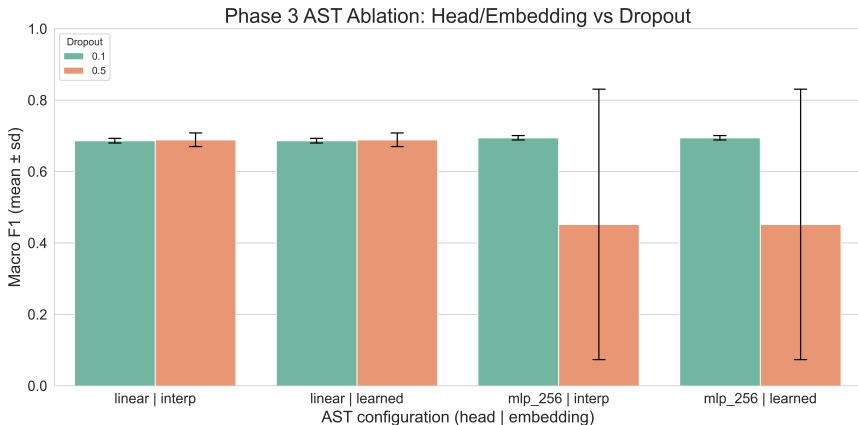
A lot slower training for xLSTM. Stable convergence for ConvNeXt across all settings, with ReduceLROnPlateau slightly faster than cosine warmup.

Phase 3: Architecture Distribution



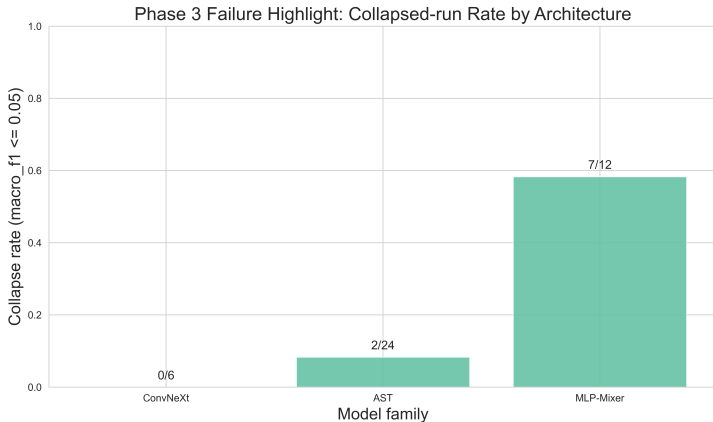
ConvNeXt is best and most stable; AST is competitive but less stable; MLP-Mixer is consistently weaker.

Phase 3: AST Ablation (Head / Embedding / Dropout)



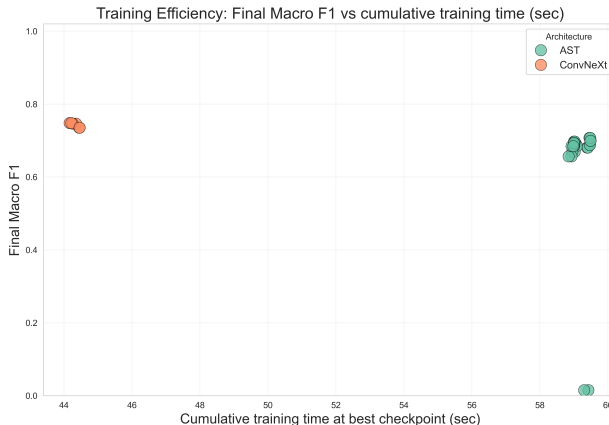
Macro F1 across AST head and embedding pairs, split by dropout; differences are moderate and best configs remain in a similar band.

Phase 3: Failure Highlight



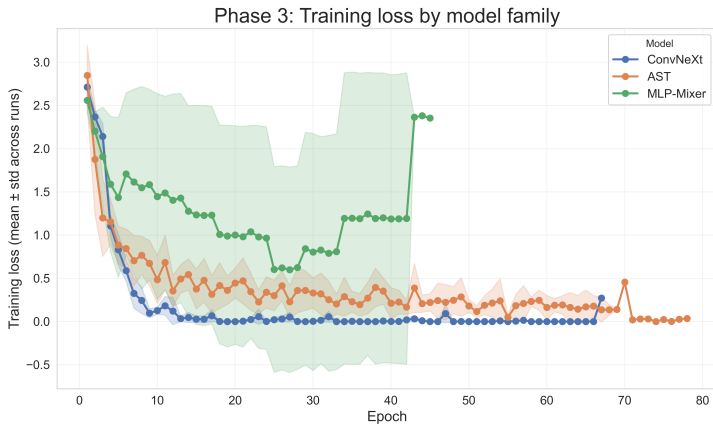
Collapse-rate analysis confirms reliability differences: ConvNeXt has the fewest severe failures, MLP-Mixer the most. Results with $F1 < 0.5$, worse than random.

Training Efficiency (ConvNeXt vs AST)



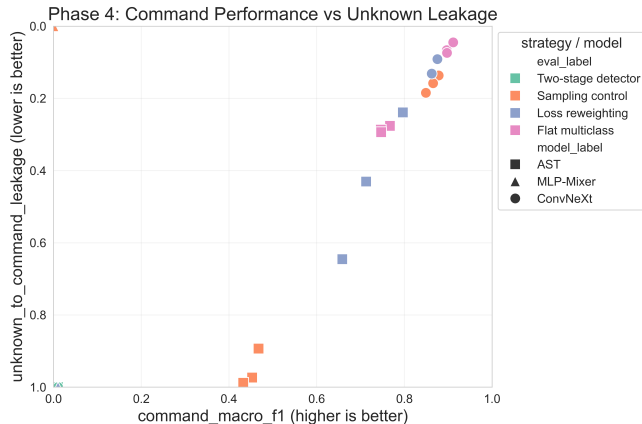
AST often converges in fewer epochs, but ConvNeXt has a lot faster total training time and better final F1.

Phase 3: Training Loss vs Epoch



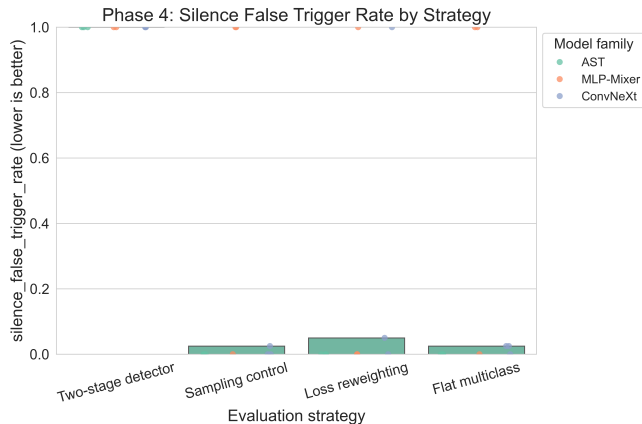
ConvNeXt shows the strongest and most consistent training loss reduction across epochs, AST shows slower and more variable convergence, while MLP-Mixer has the weakest training dynamics.

Phase 4: Open-Set Trade-off



Flat multiclass and loss reweighting dominate the command-vs-leakage frontier; two-stage detector is not competitive.

Phase 4: Silence False Trigger Rate



Two-stage detector shows a critical failure mode with elevated silence false triggers, which is unacceptable for always-on KWS. Sampling control and flat multiclass have the best silence robustness.

Phase 4: Strategy Radar



Flat multiclass delivers the strongest overall deployment profile; sampling control can help but is less consistent across models.

Per-Class Best Model Diagnostics



Residual errors concentrate on phonetically similar pairs (notably go and no), while unknown handling remains strong.

- The 4-phase funnel provides a reproducible and practical KWS evaluation protocol.
- ConvNeXt is the best overall architecture under the shared from-scratch setup.
- Flat multiclass is the safest and strongest non-command handling strategy in this benchmark.
- Unknown leakage and silence false-trigger metrics are essential for deployment decisions, however performance is good.
- Future work: explore more architectures, larger datasets, and real-world robustness tests.

Thank You

Paweł Pozorski, Paweł Florek
Warsaw University of Technology
{pawel.pozorski, pawel.florek}.stud@pw.edu.pl